Class 6

Lea Sounthong

4/15/26

Q1.Write a function add() to add some numbers together. Make sure to add comment(s) to the function explaining why you do things.

a. Your first version of the function should add two input numbers together; e.g. add(4, 7) should return 11. [1 pt]

```
add_2<-function(x,y) {
  #Adding variables x and y
  x+y
}
```

Testing my function:

```
add_2(5,7)
```

```
[1] 12
```

```
add_2(1020293,20329320)
```

```
[1] 21349613
```

Making function with R-studio help

```
add_two <- function(x, y) {
  x+y
}
```

```
add_two(x=5, y=9)
```

```
[1] 14
```

Q1 b. Your second version of the function should add any numbers together that the user inputs; e.g. add(1, 2, 3, -4) should return 2.

```
add_multi<-function(x,...){
  #Using sum because adding multiple numbers
  sum(x,...)
}

add_multi(1,2,3,-4)
```

```
[1] 2
```

```
add_multi2<-function(x,...) {
  sum(x,...)
}
```

Q2: Write a function generate_dna() that generates a random DNA sequence of a length input by the user; e.g. generate_dna(11) may return "ATTTAAGGCTG". Make sure to add comments to the function. Play with functions sample() and paste() in your R-Studio Console to see how they can help here.

```
generate_dna <- function(x) {
  #Generating a nucleotide vector
  nucs <- c("A", "T", "G","C")
  #Using sample to pull out random nucleotides
  nucs_vector <- sample (nucs, x, replace=TRUE)
  #Generating a dna string from nucs_vector
  paste(nucs_vector, collapse="")
}

#Testing my generate_data() function
generate_dna(34)
```

```
[1] "CCTCCCTTCTCTGATATGCGTTCCAAGTCTGTTC"
```

Q3: Write a function generate_protein() that generates a random peptide/protein sequence of a length input by the user; e.g. generate_protein(6) may return "WQRTAG". Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids can appear more than once. Make sure to add comments to the function.

```
generate_protein <- function(x){
  #Generating a amino acid vector with all 20 amino acids
  amino_acids <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K",
"M", "F", "P", "S", "T", "W", "Y","V")
  #Using sample to pull out random amino acids
  protein_vector <- sample(amino_acids, x, replace=TRUE)
  #Generating a protein string from protein_vector
  paste(protein_vector, collapse="")
}
#Testing my generate_protein function
generate_protein(6)
```

```
[1] "SFRWAR"
```

Q4: Use your generate_protein() function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function sapply() for this.

```
#Generating vector for peptide lengths
lengths <- c(5,7,9,11)
#Using sapply to generate peptides for each length
peptides <- sapply(lengths, function(x) generate_protein(x))
#Displaying the peptides
peptides
```

```
[1] "IYPGT"       "AKGHCIP"     "GSELKSRNW"   "GHLWIDDMWTV"
```

Note: The peptides may change every time I save the PDF after editing, but original peptides used for the BLAST searches were VFPYF, GDVPQKH, HNLKNESWE, and PEMEGKCAWMK.

Q5: Take each of your peptides from Q4 and run a BLASTP search (https://blast.ncbi.nlm.nih.gov/Blast.cgi) selecting the Non-redundant protein sequences (nr) database and homo sapiens (taxid: 9606) organism. For each search, before clicking BLAST check the Show results in a new window tab next

to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. Once each search is done, for the top aligned protein for each peptide, list the percent identity (Per. Ident) times coverage (Query Cover) (for example, in the example below, 100% Query Cover times 85.71% Per. Ident = 100% x 85.71% = 1.00 x 0.8571 x 100% = 85.71%); make sure to type out your calculations in your Quarto document [note: if any of your values are below 30% you are probably miscalculating something]. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see Data Sheet link in the Canvas assignment). Also, for each datapoint, add your initials in the "Your_Initials" column of the data sheet. Before adding your initials, scan through other used initials; if anyone has the same initials as you, modify your initials to make them unique (your initials will be used below to identify your specific data points in a ggplot graph).

```
#Peptide Sequence: VFPYF, Length_aa: 5, Query cover: 100%, Percent Identity: 100%,
#Percent Identity X Query Cover: 1.00 * 1.00 * 100 = 100%

#Peptide Sequence: GDVPQKH, Length_aa: 7, Query cover: 100%, Percent Identity: 71.43%,
#Percent Identity X Query Cover: 1.00 * 0.7143 * 100 = 71.43%

#Peptide Sequence: HNLKNESWE, Length_aa: 9, Query cover: 89%, Percent Identity: 85.71%,
#Percent Identity X Query Cover: 0.89 * 0.8571 * 100 = 76.28%

#Peptide Sequence: PEMEGKCAWMK, Length_aa: 11, Query cover: 73%, Percent Identity: 87.50%,
#Percent Identity X Query Cover: 0.73 * 0.8750 * 100 = 63.88%
```

Q6: Once there is a good amount of data in the shared excel sheet, export the sheet as a CSV UTF-8 file (Export option under the File tab) and save the file to your class working folder (i.e. the folder in which you are currently generating a Quarto document). In R-Studio, upload the csv file into a data frame using the command peptide_df <- read.csv("your_filename"). Use the head() function to list the first 10 rows (note that the default for head() is 6) of the peptide_df dataframe as a test that everything looks good.

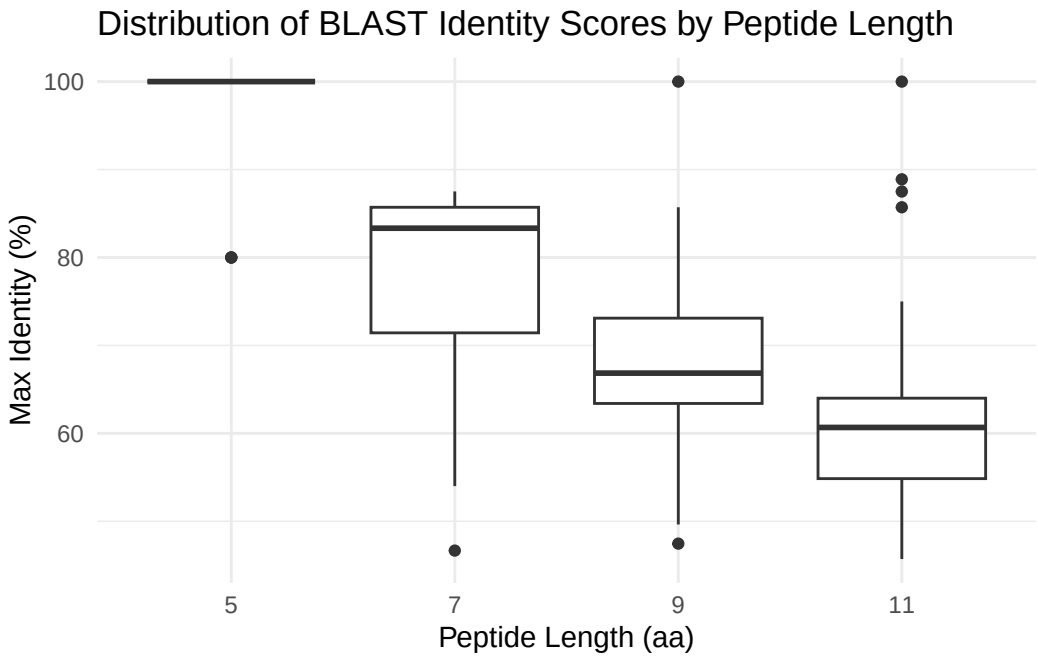
```
#Using read.csv to pull in the CSV file as a dataframe
peptide_df <- read.csv("BIMM143_peptide_FINAL.csv")
#Displaying the first 10 rows to check the data
head(peptide_df, 10)
```

```
   Length_aa Max_Identity_perc Your_Initials
1          5            100.00           JLA
2          7             85.71           JLA
```

```
3           9             66.85          JLA
4          11             55.00          JLA
5           5             80.00          SKB
6           7             85.71          SKB
7           9             66.85          SKB
8          11             64.00          SKB
9           5            100.00          KET
10          7             85.71          KET
```

Q7: Use ggplot2 to generate a box plot with Length_aa on the x-axis and Max_Identity_perc on the y-axis. Since the box plot will require categorical values for Length_aa, you may have to specify the x-axis as as.factor(Length_aa). Feel free to pretty up the plot by adding your own x/y-axis labels, graph title, etc using the labs() function and/or using your favorite theme using a theme() function (see the data-visualization_ggplot.pdf cheat sheet in Canvas for options). Optional: you will reuse this plot a few times below, so your subsequent coding will be simpler if you assign your box plot to an object such as 'p' (p <- ggplot(peptide_df) +....etc).

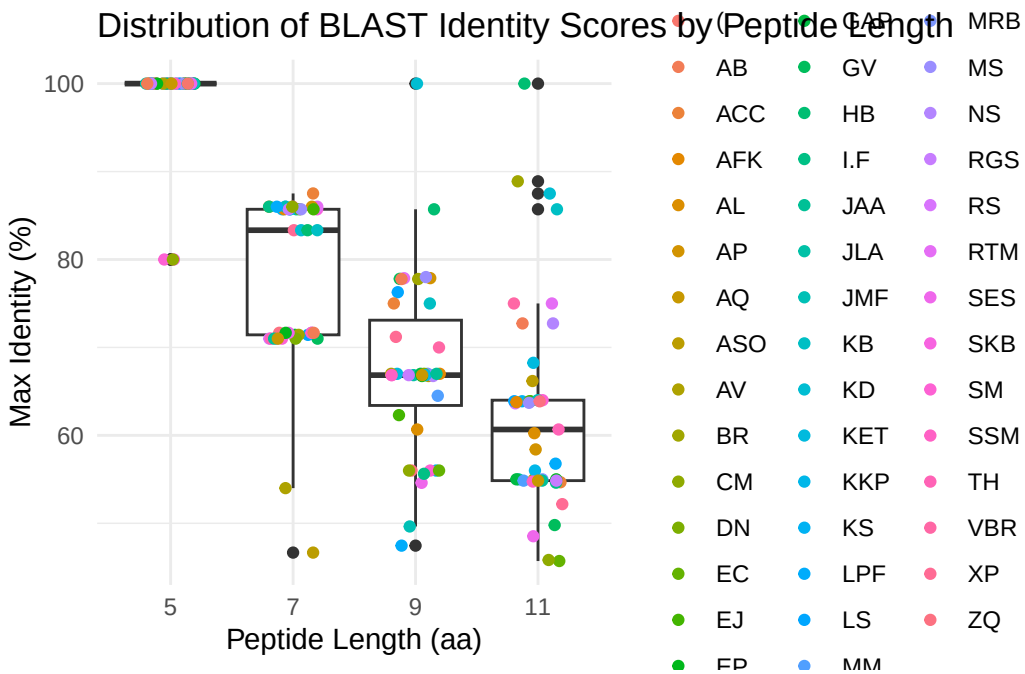
```
#Accessing ggplot2
library (ggplot2)
#Generating box plot and assigning to my_plot
my_plot <- ggplot(peptide_df, aes(x=as.factor(Length_aa), y=Max_Identity_perc)) +
  geom_boxplot() +
  labs(x="Peptide Length (aa)",
   y= "Max Identity (%)",
   title="Distribution of BLAST Identity Scores by Peptide Length") + theme_minimal()
#Testing my_plot
my_plot
```



Q8. Let's add a geom_jitter() layer to your box plot to visualize the individual data points on top of the box plot and highlight your own data points.

a. First, add a geom_jitter() layer to your box plot with each data point colored according to students' initials by adding the following additional line to your plot: + geom_jitter(aes(col=Your_Initials)). Feel free to narrow the spread of the jitter points using the width argument (e.g. width = 0.2). You can find your own datapoints this way via the color legend, but it looks rather messy.

```
#Accessing ggplot2
library (ggplot2)
#Generating box plot and assigning to my_plot
my_plot <- ggplot(peptide_df, aes(x=as.factor(Length_aa), y=Max_Identity_perc)) +
  geom_boxplot() +
  geom_jitter(aes(col=Your_Initials), width=0.2) +
  labs(x="Peptide Length (aa)",
   y= "Max Identity (%)",
   title="Distribution of BLAST Identity Scores by Peptide Length") + theme_minimal()
#Testing my_plot
my_plot
```



Q8 b. Now let's control the coloring ourselves to better highlight your own data points. We can do this by generating a vector with a color for each datapoint (we will use your favorite color for your datapoints and "GREY" for all other datapoints). You can then use this vector to assign colors with the geom_jitter() call.

i. First, use the rep() function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the peptide_df data frame. Assign this vector to an object you name color: i.e. color <- c(rep(,)) (fill in the blanks). Once you have generated the color object, use the table() function to ensure that you have the correct number of instances of "GREY" in color. [1 pt] o Hints for (i): Use your R-Studio Console to test the outputs of rep("GREY", 2), dim(peptide_df), and dim(peptide_df)[1], and make sure to look at documentation for the dim() function (?dim) to learn what the output numbers represent.

```
#Using rep to create GREY for each data point
color <- c(rep("GREY", dim(peptide_df)[1]))
#Testing the number of GREY values
table(color)
```

```
color
GREY
 172
```

ii.Next, use the Your_Initials column in the peptide_df dataframe (which can be called as peptide_df$Your_Initials) to generate a TRUE/FALSE vector indicating which datapoints are yours (TRUE) and which are not (FALSE). Assign this vector to an object you name mydata: i.e. mydata <- _____ (fill in the blank). Again, check that you have the correct number of TRUE and FALSE in mydata using the table() function.

```
#Accessing Your_Initials column
peptide_df$Your_Initials
```

```
  [1] "JLA" "JLA" "JLA" "JLA" "SKB" "SKB" "SKB" "SKB" "KET" "KET" "KET" "KET"
 [13] "RTM" "RTM" "RTM" "RTM" "GV"  "GV"  "GV"  "GV"  "RS"  "RS"  "RS"  "RS"
 [25] "LS"  "LS"  "LS"  "LS"  "TH"  "TH"  "TH"  "TH"  "KS"  "KS"  "KS"  "KS"
 [37] "XP"  "XP"  "XP"  "XP"  "SM"  "SM"  "SM"  "SM"  "AP"  "AP"  "AP"  "AP"
 [49] "ASO" "ASO" "ASO" "ASO" "MRB" "MRB" "MRB" "MRB" "VBR" "VBR" "VBR" "VBR"
 [61] "GAP" "GAP" "GAP" "GAP" "MM"  "MM"  "MM"  "MM"  "BR"  "BR"  "BR"  "BR"
 [73] "AFK" "AFK" "AFK" "AFK" "KD"  "KD"  "KD"  "KD"  "NS"  "NS"  "NS"  "NS"
 [85] "DN"  "DN"  "DN"  "DN"  "EJ"  "EJ"  "EJ"  "EJ"  "HB"  "HB"  "HB"  "HB"
 [97] "AV"  "AV"  "AV"  "AV"  "ACC" "ACC" "ACC" "ACC" "JMF" "JMF" "JMF" "JMF"
[109] "SES" "SES" "SES" "SES" "AL"  "AL"  "AL"  "AL"  "EC"  "EC"  "EC"  "EC"
[121] "MS"  "MS"  "MS"  "MS"  "KKP" "KKP" "KKP" "KKP" "KB"  "KB"  "KB"  "KB"
[133] "ZQ"  "ZQ"  "ZQ"  "ZQ"  "I.F" "I.F" "I.F" "I.F" "EP"  "EP"  "EP"  "EP"
[145] "JAA" "JAA" "JAA" "JAA" "SSM" "SSM" "SSM" "SSM" "AQ"  "AQ"  "AQ"  "AQ"
[157] "RGS" "RGS" "RGS" "RGS" "CM"  "CM"  "CM"  "CM"  "("   "LPF" "LPF" "LPF"
[169] "AB"  "AB"  "AB"  "AB"
```

```
#Using initials to identify my data points
mydata <- peptide_df$Your_Initials=="LS"
#Using table with TRUE and FALSE to see how many data points I have
table(mydata)
```

```
mydata
FALSE  TRUE
  168     4
```

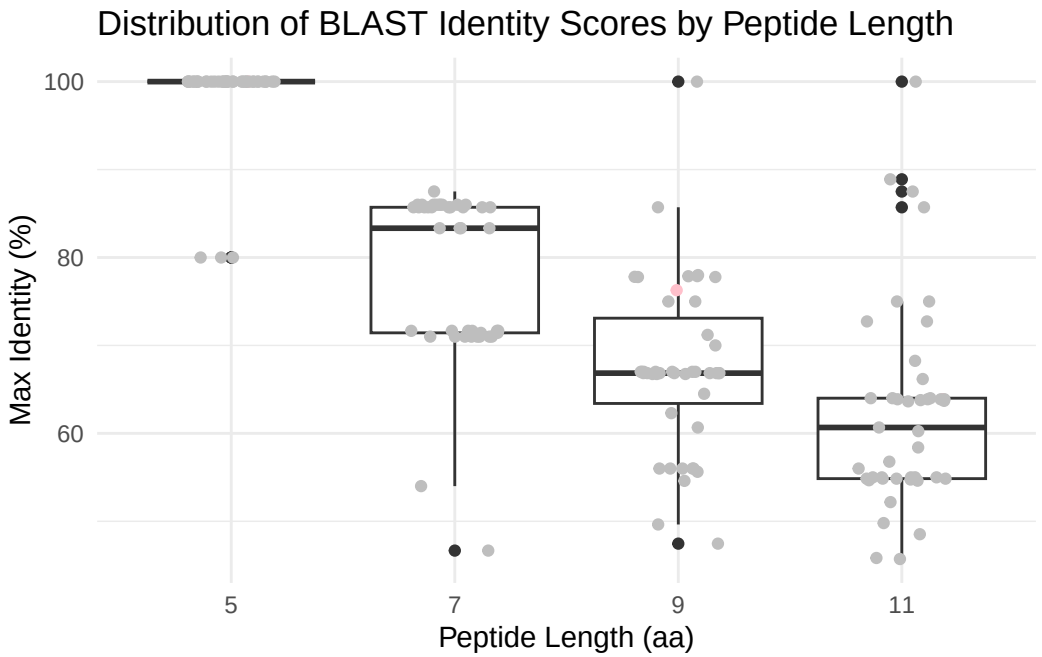
iii. Now, use the mydata vector to replace the "GREY" values in the color vector that correspond to your data points with your favorite color, like this: color[mydata] <- "COLOR" (replace "COLOR" with the name of your favorite color; see link at the bottom of the page to a webpage with 657 R color names to select a cool color). Note, this will replace "GREY" with your favorite color for all the positions that are TRUE in mydata, i.e. the positions of your data points. Again, check the vector with table().

```r
#Using mydata to change my points to pink
color[mydata] <- "pink"
#Testing updated color values
table(color)
```

```
color
GREY pink
 168    4
```

iv.Now you are ready to remake the plot by adding the following line to your original box plot from Q7: + geom_jitter(col=color). (Note that the color call is not run as an aesthetic here because we are now directly telling ggplot what color to use for each dot). Again, feel free to use the width argument to narrow the jitter points if you wish (e.g. width = 0.2).

```r
#Generating final box plot and assigning to my_plot
my_plot <- ggplot(peptide_df, aes(x=as.factor(Length_aa), y=Max_Identity_perc)) +
  geom_boxplot() +
  geom_jitter(col=color, width=0.2) +
  labs(x="Peptide Length (aa)",
   y= "Max Identity (%)",
   title="Distribution of BLAST Identity Scores by Peptide Length") + theme_minimal()
#Testing my_plot
my_plot
```



Q8 c. Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s).

Yes, one of my data points, the peptide with length 9 aa is above the 75th percentile of the class data.

Q9: MHC class II molecules of our immune system display 9 amino acid peptides on the surface of immune cells that, when from a foreign origin, activates a T-cell immune response. Based on your findings in Q7 and 8, speculate why those peptides are 9 amino acids long rather than, say, 5 amino acids.

As demonstrated in the boxplot, 9 amino acid peptides have lower identity and greater variability than 5 amino acid peptides, which tend to have higher identity and less variability. Therefore, 9 amino acid peptides are more distinctive, and this increased specificity helps the immune system distinguish foreign pathogens from self. MHC class II molecules display 9 amino acid peptides because this level of distinctiveness allows for more accurate immune recognition.